

ELEC 475: Computer Vision

Lab 3

Jacob Chisholm (21jc138)

November 10, 2025

Contents

1	Network Architecture	1
1.1	Encoder	1
1.2	Context Module	1
1.3	Decoder	1
2	Knowledge Distillation	1
2.1	Response Based Learning	2
2.2	Feature Based Learning	2
3	Training	3
3.1	Direct Training	3
3.2	Response Based Training	4
3.3	Feature Based Training	5
4	Experiments	6
5	Performance	8
6	LLM	8

List of Figures

1	200 Epoch Direct Training	3
2	2000 Epoch Direct Training	4
3	100 Epoch Response-Based Training	4
4	1000 Epoch Response-Based Training	5
5	500 Epoch Feature-Based Training	5
6	ResNet50 Sample Image	6
7	Direct Trained Model Sample Image	7
8	Response Trained Model Sample Image	7
9	Feature Trained Model Sample Image	7

List of Tables

1	Testing Results	6
2	Comparison to ResNet50	6

1 Network Architecture

The segmentation model is a lightweight encoder-decoder network based on the MobileNetV3-Small architecture. It uses an Atrous Spacial Pyramid Pooling (ASPP) context module and follows the standard encoder-context-decoder pattern.

1.1 Encoder

The encoder uses the convolution based feature extractor from MobileNetV3-Small pretrained on ImageNet. Input images of size $224 \times 224 \times 3$ are progressively down-sampled to produce 576 output channels. Three feature taps are extracted at layers 2, 3, and 12.

1.2 Context Module

ASPP is used as the context module with 576 input channels and 256 output channels. The module consists of four parallel 3×3 convolutions followed by a 1×1 convolution. All five outputs are concatenated along the channel dimensions, projected to the 256 output channels. The ASPP output is normalized, activated with ReLU, and regularized with a 0.5 dropout.

1.3 Decoder

To recover spatial precision, the ASPP output is upsampled to match the low-level feature resolution. The low-level features are first projected from 24 to 48 channels using a 1×1 convolution, batch normalization, and ReLU activation. The two tensors are then concatenated ($48 + 256$ channels) and refined through a 3×3 convolution giving 256 channels. After batch normalization and ReLU activation, a final 1×1 convolution is used to produce the final output.

2 Knowledge Distillation

Knowledge Distillation is the process of using a large pretrained neural network to help train a smaller neural network. In knowledge distillation, the smaller model attempts to learn the outputs of the larger model. This can help the smaller model learn faster and generalize better.

In this lab, both response-based and feature-based learning were employed to replicate the behavior of ResNet50 in the smaller segmentation model described in Section 1.

2.1 Response Based Learning

Response-based learning is when the "student" model attempts to match the outputs of the "teacher" model. In this method, the student and teacher models are given the same batches of input data. The outputs of the models are then compared against each other and the ground truth to find the total loss. The function used for comparison (shown below) contains three new hyperparameters.

$$\mathcal{L}(x; W) = \alpha * \mathcal{H}(y, \sigma(z_s; T = 1)) + \beta * \mathcal{H}(\sigma(z_t; T = \tau), \sigma(z_s, T = \tau))$$

In essence, this function is a weighted combination of the differences between the student (z_s) and teacher (z_t) models, as well as the difference between the models and ground truth (y). This function is used to generate the loss backpropagated through the student model. The teacher model is purely in evaluation mode and is not trained during model distillation.

The three new hyperparameters α , β , and τ , are used to control the ratio of the loss function. α is used to control how much the difference between the model (z_s) and ground truth (y) impacts the loss function. β is used to control how much the model attempts to replicate the output of the teacher. Finally, τ is the temperature. This parameter is used in the modified softmax function and controls how "soft" the model outputs are.

To better understand the temperature value, the modified softmax function is shown below.

$$p_i(T) = \frac{\exp\left(\frac{z_i}{T}\right)}{\sum_{j=1}^C \exp\left(\frac{z_j}{T}\right)}$$

Using a higher temperature value is best when the teacher model is very confident in its results, as it will lower this confidence value and expose more information about the other classes. However, if the temperature value is too high, it can cause the distribution to become uniform, and the student will stop learning meaningful differences.

2.2 Feature Based Learning

Feature based knowledge distillation is similar to the response based method, but rather than only involving the outputs, it involves several layers in both networks. By extracting and comparing internal layers between the models, hidden features and understandings can be transferred at a deeper level. This can help the student model to learn things like edges, textures and more abstract representations of objects.

For each internal layer extracted from both models, the layers are reshaped from (B, C, H, W) to

$(B*H*W, C)$. This treats spacial location as an independent vector of size C (the number of channels), allowing the internal layers to be compared. Without reshaping the images, the student model would only learn the global average, defeating the point of feature based learning.

The internal layers are compared using the cosine similarity via a cosine embedding loss. Each feature vector in the student is encouraged to point in the same direction as the vector in the teacher. This is done independently for each layer, and the results are summed.

Finally, this loss is added to the standard loss function through a weighted sum.

$$L_{total} = \alpha \times L_{task} + \beta * L_{feature}$$

Again, α and β are additional hyperparameters used in the tuning process. In this case, α controls how much the ground truth error influences the loss value. β controls how much the difference between the teacher and student affects the loss value.

3 Training

All training was performed on an RTX3080 GPU. The neural network was first trained directly on the dataset to establish a baseline. The neural network was then trained using both response-based and feature-based learning, where it attempted to learn from the larger ResNet50 network.

3.1 Direct Training

Using a learning rate of 0.03, batch size of 16, and a decay rate of 0.003, the model was trained on the data set over 200 epochs yielding the training plot shown in Figure 1.

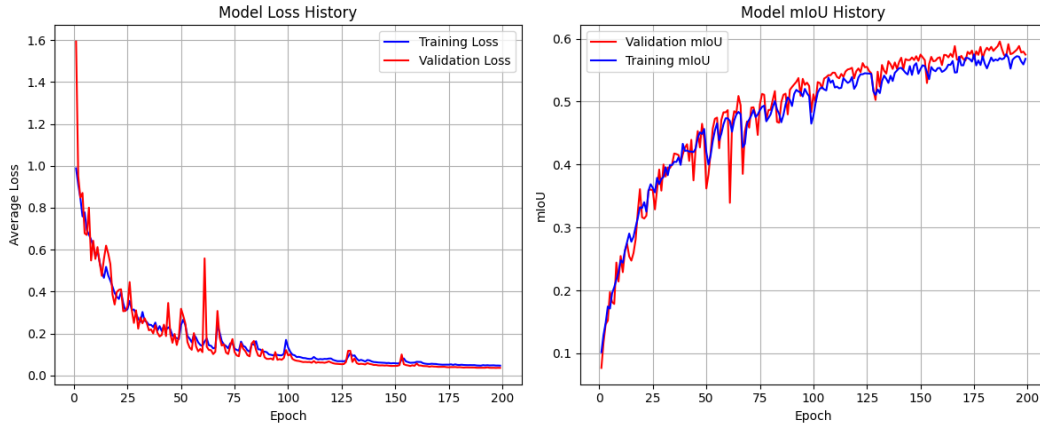


Figure 1: 200 Epoch Direct Training

This training achieved an error rate of 0.046 with a mean intersection over union (mIoU) of 0.56. At this point, the model still showed signs of improvement, so it was trained for an additional 1800 epochs.

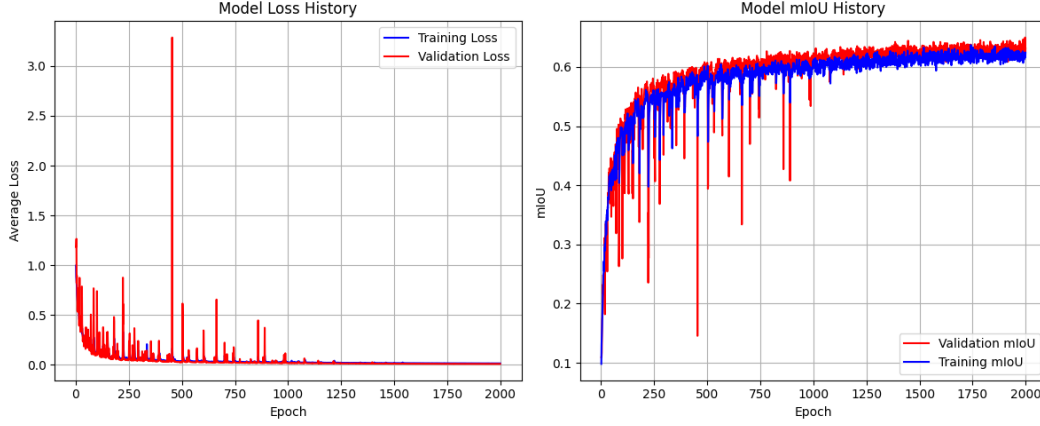


Figure 2: 2000 Epoch Direct Training

The additional training yielded a final accuracy of 0.013 with a mIoU of 0.83 after approximately 6 hours of training. Note that the mIoU in Figure 2 does not match this number. This was due to a miscalculation of the metric during training.

3.2 Response Based Training

Using the same learning rates as direct learning, an α value of 0.9, β of 0.2, and a temperature value of 0.9, the model was trained for 100 epochs. The training plot for this training is shown in Figure 3.

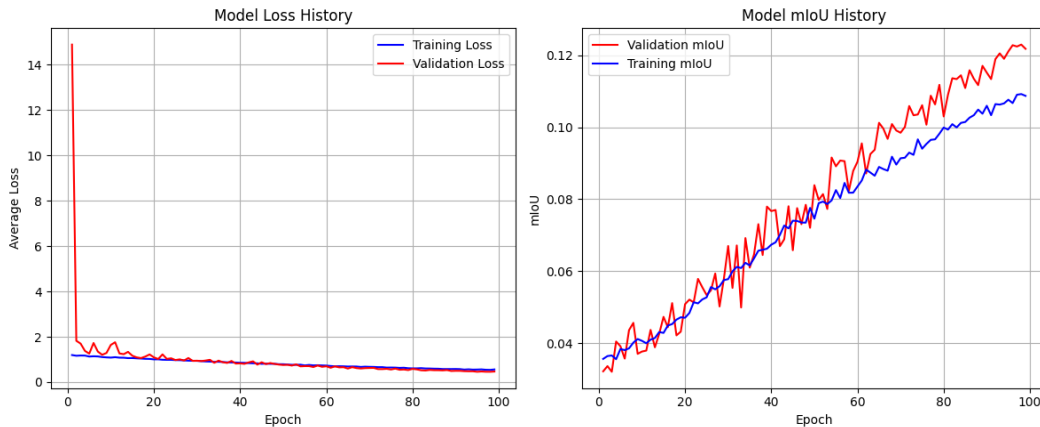


Figure 3: 100 Epoch Response-Based Training

The initial training yielded an error rate of 0.56 and was still showing signs of improvement. Therefore, the model was trained for an additional 1000 epochs. This additional training is shown in Figure

4.

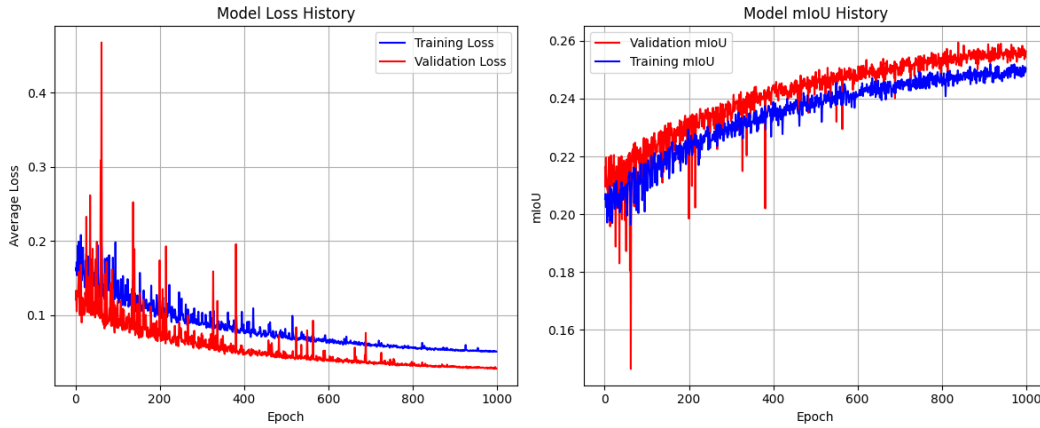


Figure 4: 1000 Epoch Response-Based Training

After approximately 9 hours of training, the final error rate was 0.05 and the mIoU was 0.81. Note that this does not match the result shown in the figure due to a calculation error in the training loop.

3.3 Feature Based Training

Using the same learning rates as the previous two training methods, $\alpha = 0.8$, $\beta = 0.2$, and temperature=1, the model was trained using feature based learning over 500 epochs. The training plot is shown in Figure 5.

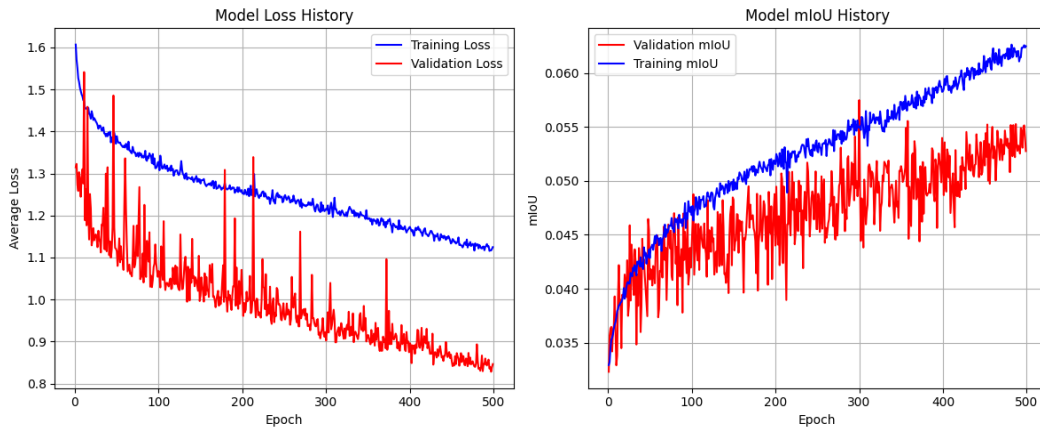


Figure 5: 500 Epoch Feature-Based Training

After 11 hours of training, the model achieved an error rate of 0.21 with a mIoU of 0.63. Note that Figure 5 does not reflect this due to an error in the training calculations.

4 Experiments

All experiments were conducted using an RTX3080 GPU. Comparisons between learning methods are shown in Table 1. Inference speed is measured in images/second. Mean Intersection over Union (mIoU) is the measurement of overlap between the predicted segments and the ground truth labels. An mIoU of 1 is considered optimal, while an mIoU of 0 is considered unacceptable.

Learning Method	mIoU	# Parameters	Inference Speed
Direct	0.83	7421205	179.2
Response Based	0.81	7421205	182.5
Feature Based	0.63	7421205	179.6

Table 1: Testing Results

Comparisons between the trained models and ResNet50 are shown in Table 2.

Model	# Parameters	Loss	mIoU	Inference Speed
MobileNetV3 - Direct	7,421,205	0.01	0.83	197.2
MobileNetV3 - Response	7,421,205	0.05	0.81	182.5
MobileNetV3 - Feature	7,421,205	0.21	0.63	179.6
ResNet50	35,322,218	0.08	0.67	85.1

Table 2: Comparison to ResNet50

Figures 6, 7, 8, and 9 show sample outputs from the models next to the ground truth labels.

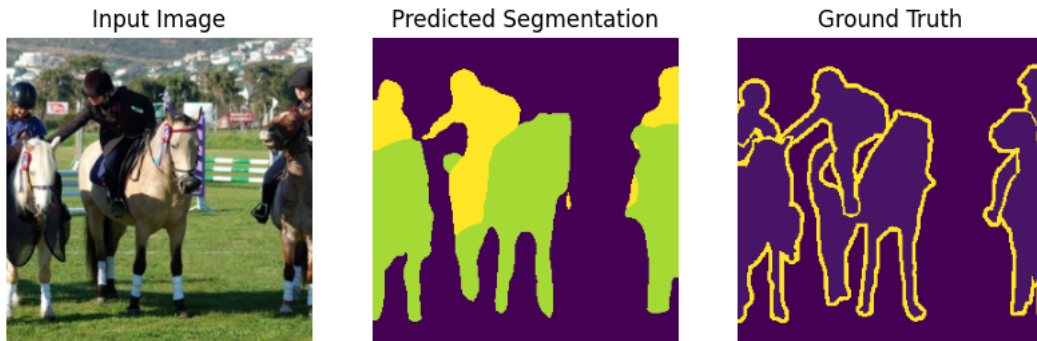


Figure 6: ResNet50 Sample Image

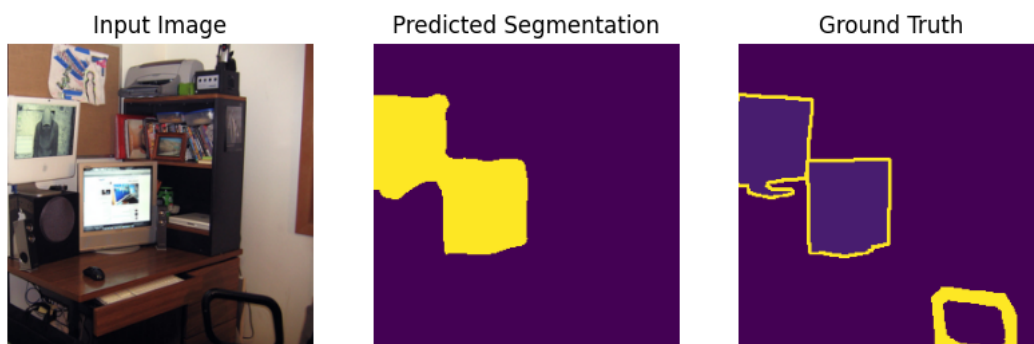


Figure 7: Direct Trained Model Sample Image

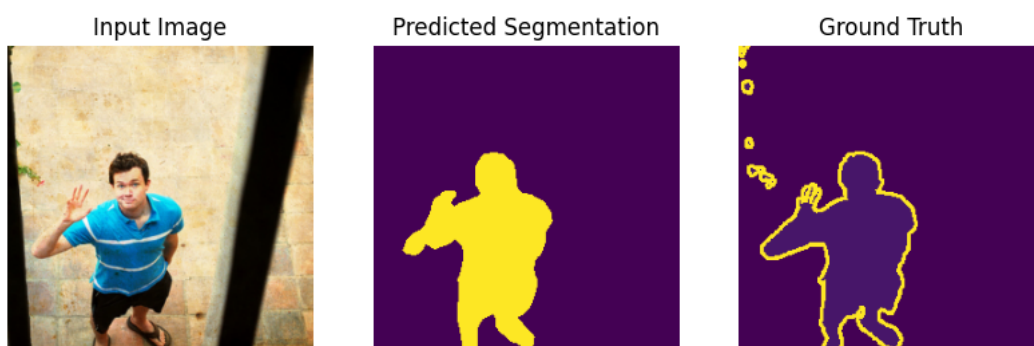


Figure 8: Response Trained Model Sample Image

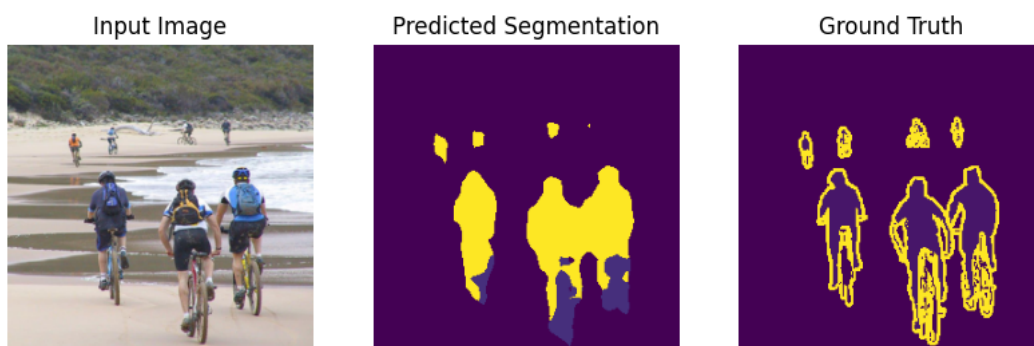


Figure 9: Feature Trained Model Sample Image

5 Performance

The system performed exceedingly well. The smaller model outperformed ResNet50 with only a fraction of the number of parameters. Given the performance disparity, knowledge distillation yielded little improvement. In the case of feature distillation, it even caused degradation in the mIoU metric. This is likely due to the smaller model starting to register smaller details in the image that were previously ignored.

Data Augmentation was immensely helpful in the training of this model. The data was augmented using both randomized flips and crops of the training images. Initial results were trained without data augmentation and performed far worse than the models presented, which used data augmentation.

In developing this system, several challenges were faced including incorrect mIoU readings during training, and image transformations being applied incorrectly. The underlying reason for mIoU readings being calculated incorrectly during training is still unknown. Initially, image transformations were being applied differently to both the input and label images. This was solved through the creation of a custom dataloader able to apply the same randomly generated transformation to both images.

6 LLM

Large Language Models were not used in the creation of this code. Rather, they were used to debug logical issues and help guide understanding while implementing modules. Several LLMs were used, including Gemini and ChatGPT.

References

- [1] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1314–1324.
- [2] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder-decoder with atrous separable convolution for semantic image segmentation," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 801–818.
- [3] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," *arXiv preprint arXiv:1706.05587*, 2017.